МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИТ)
Кафедра инструментального и прикладного программного обеспечения (ИиППО)

КУРСОВАЯ РАБОТА

по дисциплине: Проектирование и разработка серверных частей интернет-ресурсов

по профилю: Разработка и дизайн компьютерных игр и мультимедийных приложений

направления профессиональной подготовки: 09.03.04 «Программная инженерия»

Тема: Серверная часть веб-приложения «Расписание»

Студент: Васильев Борис Александрович

Группа: ИКБО-20-23

Работа представлена к защите 23.12.2025 _____ /Васильев Б.А./

(подпись и ф.и.о. студента)

Руководитель: старший преподаватель Романченко А.Е.

Работа допущена к защите 23.12.25 _____ /Романченко А.Е./

(подпись и ф.и.о. рук-ля)

Оценка по итогам защиты: ОТЛ.

_____ 23.12.25 / Романченко А.Е., ст. преп. /

_____ 23.12.25 / Гамбарян Р.Г., к. т. н., доц. /

(подписи, дата, ф.и.о., должность, звание, уч. степень двух преподавателей, проводивших защиту курсовой работы)

М. РТУ МИРЭА. 2025

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИТ)
Кафедра инструментального и прикладного программного обеспечения (ИиППО)

ЗАДАНИЕ
на выполнение курсовой работы

по дисциплине: Проектирование и разработка серверных частей интернет-ресурсов
по профилю: Разработка и дизайн компьютерных игр и мультимедийных приложений
направления профессиональной подготовки: Программная инженерия (09.03.04)

Студент: Васильев Борис Александрович
Группа: ИКБО-20-23
Срок представления к защите: 12.12.2025
Руководитель: Романченко Алексей Евгеньевич, старший преподаватель

Тема: Серверная часть веб-приложения «Расписание»

Исходные данные: используемые технологии: HTML5, CSS3, PHP, Visual Studio Code, SQL СУБД, наличие: межстраничной навигации, внешнего вида страниц, соответствующего современным стандартам веб-разработки, использование паттерна проектирования MVC. Нормативный документ: инструкция по организации и проведению курсового проектирования СМКО МИРЭА 7.5.1/04.И.05-18.

Перечень вопросов, подлежащих разработке, и обязательного графического материала: 1. Провести анализ предметной области разрабатываемого веб-приложения. 2. Обосновать выбор технологий разработки веб-приложения. 3. Разработать архитектуру веб-приложения на основе выбранного паттерна проектирования. 4. Реализовать слой серверной логики веб-приложения с применением выбранной технологии. 5. Реализовать слой логики базы данных. 6. Разработать слой клиентского представления веб-приложения 7. Создать презентацию по выполненной курсовой работе.

Руководителем произведён инструктаж по технике безопасности, противопожарной технике и правилам внутреннего распорядка.

Зав. кафедрой ИиППО: _____/Р. Г. Болбаков/, «14» 09 2025 г.

Задание на КР выдал: _____/А.Е. Романченко/, «15» сентября 2025 г.

Задание на КР получил: _____/Б.А. Васильев/, «15» сентября 2025 г.

УДК 004

Васильев Б.А.

Курсовая работа по профилю «Разработка и дизайн компьютерных игр и мультимедийных приложений» направления подготовки 09.03.04. «Программная инженерия» на тему «Серверная часть веб-приложения «Расписание»» (5-й семестр) / руководитель ст. преп. Романченко А.Е. / кафедра ИиППО Института ИТ РТУ МИРЭА – 42 стр., 26 рис., 8 лист. 26 ист., 1 прил.

Целью данной курсовой работы является проектирование и разработка серверной части веб-приложения «Расписание».

В результате выполнения работы анализ предметной области разрабатываемого веб-приложения, обоснован выбор технологий разработки, разработана архитектура веб-приложения на основе выбранного паттерна проектирования, реализован слой серверной логики веб-приложения с применением выбранной технологии, реализован слой логики базы данных, разработан слой клиентского представления веб-приложения, создана презентация по выполненной курсовой работе.

Vasilyev B.A.

Course work in the profile "Development and Design of Computer Games and Multimedia Applications" of the direction of bachelor education 09.03.04 "Software Engineering" (5th semester) / Supervisor Senior Lecturer Romanchenko A.E. / Department of IIPPO Institute of IT RTU MIREA – 42 p., 26 fig., 8 listings, 26 sources, 1 annex.

The purpose of this course work is to design and develop the server part of the web application "Schedule".

As a result of the work, the subject area of the developed web application was analyzed, the choice of development technologies was justified, the architecture of the web application based on the selected design pattern was developed, the server logic layer of the web application was implemented using the chosen technology, the database logic layer was implemented, the client presentation layer of the web application was developed, and a presentation on the completed course work was created.

М. МИРЭА. Ин-т ИТ. Каф. ИиППО. 2025г. © Б.А. Васильев

СОДЕРЖАНИЕ

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ, ОБОЗНАЧЕНИЙ И ТЕРМИНОВ 5

ВВЕДЕНИЕ..... 6

ОСНОВНАЯ ЧАСТЬ..... 7

1 Анализ предметной области 7

1.1 Общая характеристика и актуальность..... 7

1.2 Анализ существующих аналогов..... 7

1.3 Определение требований к приложению 10

2 Проектирование веб-приложения 12

2.1 Проектирование архитектуры веб-приложения..... 12

2.2 Выбор и обоснование используемых технологий 13

2.3 Проектирование структуры базы данных..... 14

2.4 Проектирование пользовательского интерфейса и навигации 16

3 Разработка веб-приложения..... 18

3.1 Конфигурация среды выполнения..... 18

3.2 Реализация слоя базы данных..... 18

3.3 Реализация слоя серверной логики 21

3.4 Реализация слоя клиентского представления 26

4 Функциональное тестирование веб-приложения 31

4.1 Ход тестирования..... 31

4.2 Итоги тестирования 38

ЗАКЛЮЧЕНИЕ 39

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 40

ПРИЛОЖЕНИЕ..... 43

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ, ОБОЗНАЧЕНИЙ И ТЕРМИНОВ

В отчёте используются следующие сокращения обозначения и термины.

Active Record	– паттерн проектирования, связывающий объект в памяти с записью в базе данных, упрощая операции CRUD
AJAX	– технология асинхронного обмена данными с сервером без перезагрузки страницы
CRUD	– четыре базовые операции с данными: создание, чтение, обновление, удаление
HTTP	– протокол передачи гипертекста, используемый для обмена данными во всемирной сети
JSON	– легковесный текстовый формат обмена данными, основанный на синтаксисе объектов в JavaScript
MVC	– архитектурный паттерн, разделяющий приложение на модель, представление и контроллер
UML	– унифицированный язык моделирования, используемый для визуализации, спецификации, конструирования и документирования систем
URL	– унифицированный указатель ресурса, опредекляющий местонахождение ресурса в интернете
Клиент	– аппаратное или программное обеспечение, запрашивающее ресурсы или услуги у сервера
Модальное окно	– элемент пользовательского интерфейса, блокирующий взаимодействие с основным содержимым страницы до его закрытия
Сервер	– аппаратное или программное обеспечение, предоставляющее ресурсы или услуги клиентам
Хедер	– блок в верхней части страницы, обеспечивающий навигацию и доступ к ключевой информации сайта

ВВЕДЕНИЕ

В условиях современного ритма жизни эффективное планирование становится необходимым навыком для достижения личных и профессиональных целей. Существующие цифровые инструменты часто предлагают либо избыточный функционал для корпоративного управления проектами, либо слишком упрощенный подход в виде обычных списков дел. Это создает потребность в специализированных решениях, сфокусированных на конкретной задаче — удобном управлении персональным, циклично повторяющимся недельным расписанием.

Целью данной курсовой работы является проектирование и разработка серверной части веб-приложения «Расписание».

Для достижения поставленной цели необходимо решить следующий ряд задач:

– провести анализ предметной области разрабатываемого веб-приложения,

– обосновать выбор технологий разработки веб-приложения,

– разработать архитектуру веб-приложения на основе выбранного паттерна проектирования,

– реализовать слой логики базы данных,

– реализовать слой серверной логики веб-приложения,

– разработать слой клиентского представления веб-приложения.

Объектом исследования является веб-приложение для управления персональным расписанием на неделю.

Предметом исследования является процесс проектирования и разработки серверной части веб-приложения для управления персональным расписанием на неделю.

ОСНОВНАЯ ЧАСТЬ

1 Анализ предметной области

1.1 Общая характеристика и актуальность

В современном мире, характеризующемся высокой динамикой и многозадачностью, эффективное управление личным временем становится ключевым фактором продуктивности. Повседневная жизнь требует систематизации множества повторяющихся активностей, таких как учебные занятия, рабочие обязанности и личные хобби.

Решить эту задачу призваны специализированные приложения, помогающие визуализировать и структурировать недельную рутину. В отличие от сложных календарных систем, ориентированных на конкретные даты, или таск-менеджеров, сфокусированных на выполнении отдельных задач, эти инструменты делают акцент на управлении повторяющимися событиями в рамках недели. Их основная цель — предоставить пользователю ясный и удобный обзор своего регулярного расписания, способствуя формированию организованного образа жизни.

1.2 Анализ существующих аналогов

Для решения задачи составления расписания существует множество инструментов. В рамках анализа были рассмотрены три различных по своему подходу аналога: графические редакторы Visme [1] и Planzeo [2], и узкоспециализированный программный комплекс Gensio [3].

Visme представляет собой графический онлайн-редактор, предлагающий обширную библиотеку шаблонов для создания различных видов визуального контента, включая расписания. Однако его основной фокус — это дизайн и визуализация, что означает работу с расписанием как со статичным изображением или документом, а не как со структурированными данными, что ограничивает возможности по их валидации и фильтрации (рисунок 1.1).

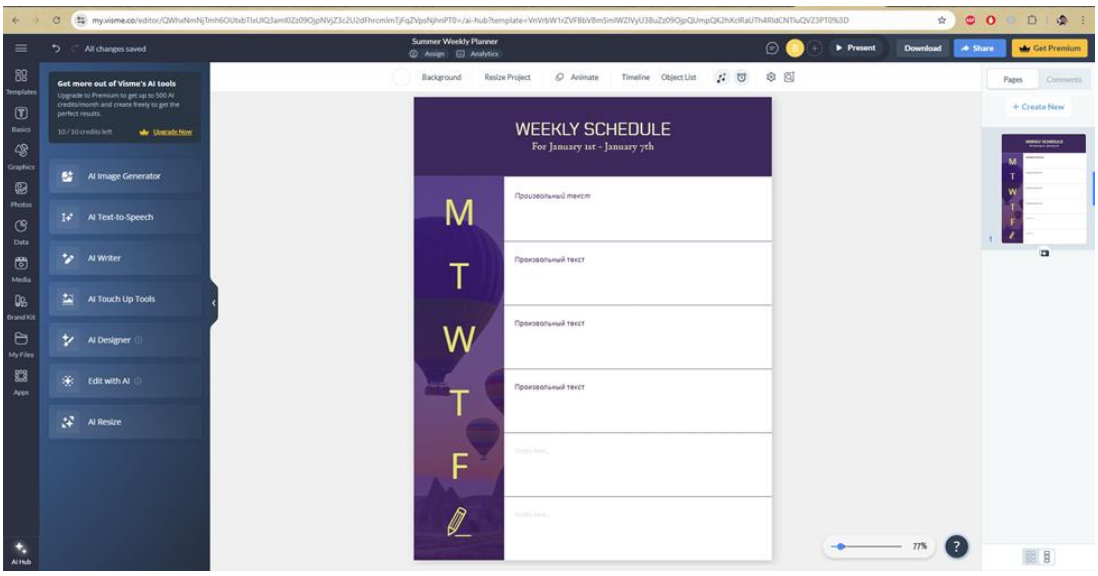

Рисунок 1.1 — Скриншот интерфейса сервиса Visme

Planzeo, хоть и более специализирован, чем Visme, предлагая редактирование расписания в табличном формате, всё же работает с записями как с простым текстом. Это означает отсутствие какой-либо валидации данных (например, имеется возможность ввести произвольный текст в столбец для времени). Также по какой-то причине нет возможности добавить в расписание записи на воскресенье (рисунок 1.2).

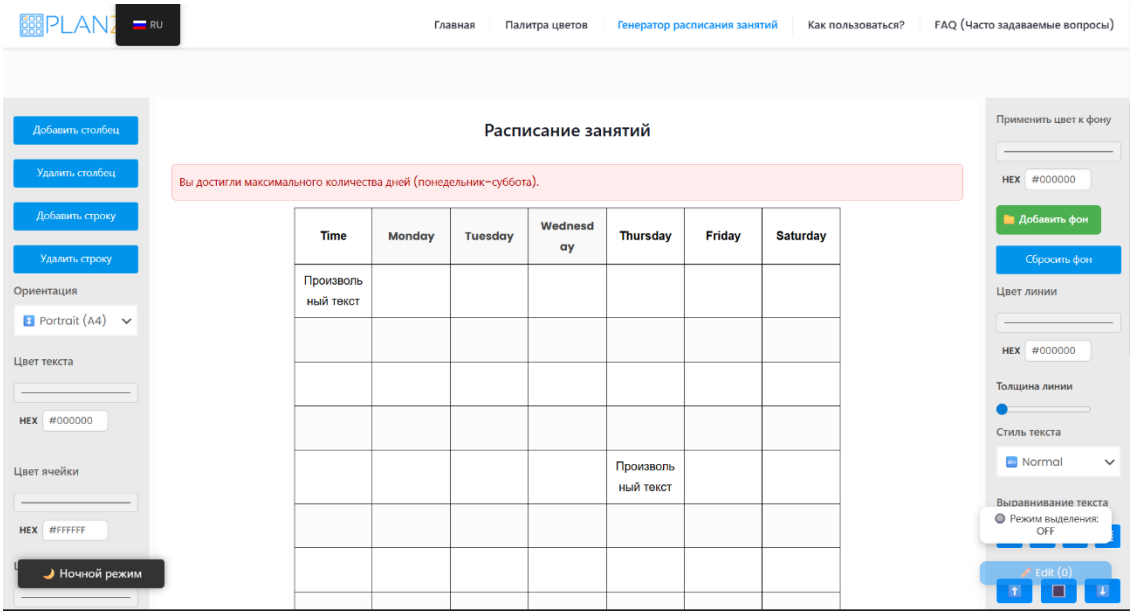

Рисунок 1.2 — Скриншот интерфейса сервиса Planzeo

Gensio представляет собой мощный программный комплекс, предоставляющий помимо прочего функционал автоматической генерации сложных расписаний для учебных заведений. Он учитывает множество специфических параметров, таких как нагрузка преподавателей, доступность аудиторий и нормы СанПиН. Вследствие своей узкой специализации и сложности, данный инструмент является избыточным и не подходит для обычного пользователя, которому требуется простое средство для организации личного недельного расписания (рисунок 1.3).

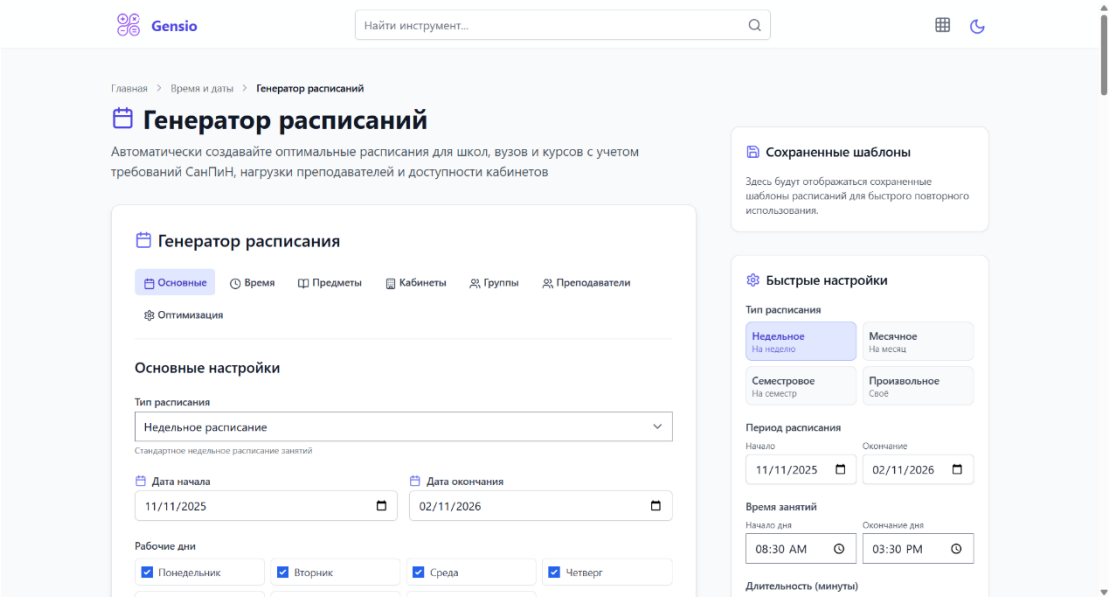

Рисунок 1.3 — Скриншот интерфейса сервиса Gensio

Таким образом, проведенный анализ показывает, что на рынке существует незанятая ниша. С одной стороны, имеются простые графические инструменты, которые не обеспечивают работу с расписанием как со структурированными данными. С другой — сложные программные комплексы, избыточные для личного пользования. Это обосновывает актуальность разработки веб-приложения, которое позволит пользователю просто и удобно управлять личным недельным расписанием, работая с записями как с полноценными объектами данных, а не просто с текстом или графикой.

1.3 Определение требований к приложению

Ключевыми требованиями, вытекающими из анализа аналогов, являются ориентация на простоту и удобство для личного пользования, а также работа с расписанием как со структурированными данными. Первый принцип отличает приложение от избыточно сложных программных комплексов, а второй — от графических редакторов, что является основой для реализации валидации вводимых данных и гибкой фильтрации.

Ниже представлен перечень ключевых возможностей, которые должно предоставлять разрабатываемое приложение:

– возможность аутентификации пользователя, включая создание учётной записи, вход в систему и выход из неё,

– возможность просмотра расписания в двух режимах: на всю неделю и на отдельный день,

– возможность управления записями в расписании (создание, чтение, изменение, удаление),

– возможность управления персонализированными тегами (создание, чтение, изменение, удаление),

– возможность прикрепления тегов к записям в расписании и открепления тегов от записей в расписании,

– возможность фильтрации расписания по тексту записей и по прикреплённым тегам.

Для наглядного представления взаимодействия пользователя с приложением была разработана UML-диаграмма вариантов использования [4] (рисунок 1.4).

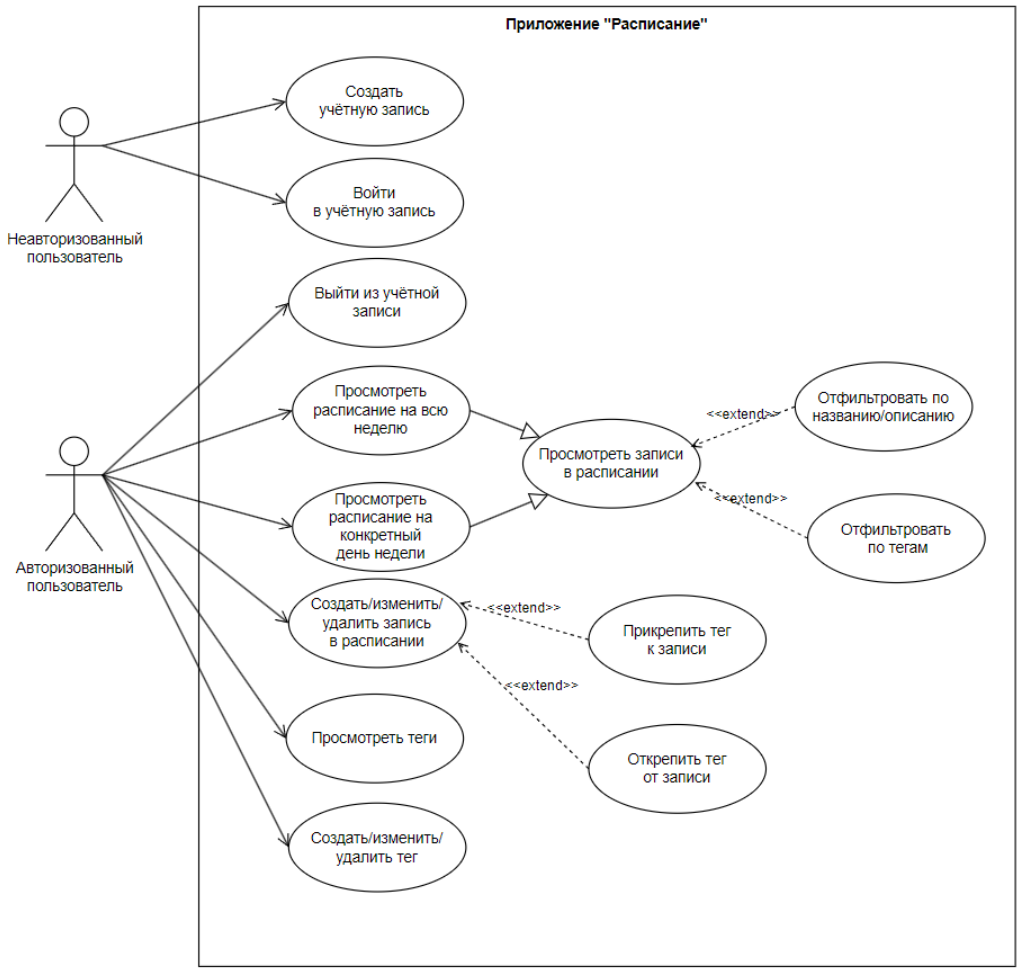

Рисунок 1.4 – UML-диаграмма вариантов использования приложения

2 Проектирование веб-приложения

2.1 Проектирование архитектуры веб-приложения

Для веб-приложения была выбрана классическая трёхзвенная клиент-серверная архитектура, являющаяся отраслевым стандартом для подобных систем [5]. Эта архитектура разделяет приложение на три основных звена, что обеспечивает гибкость разработки, независимость компонентов и масштабируемость. Взаимодействие этих звеньев представлено на рисунке 2.1.

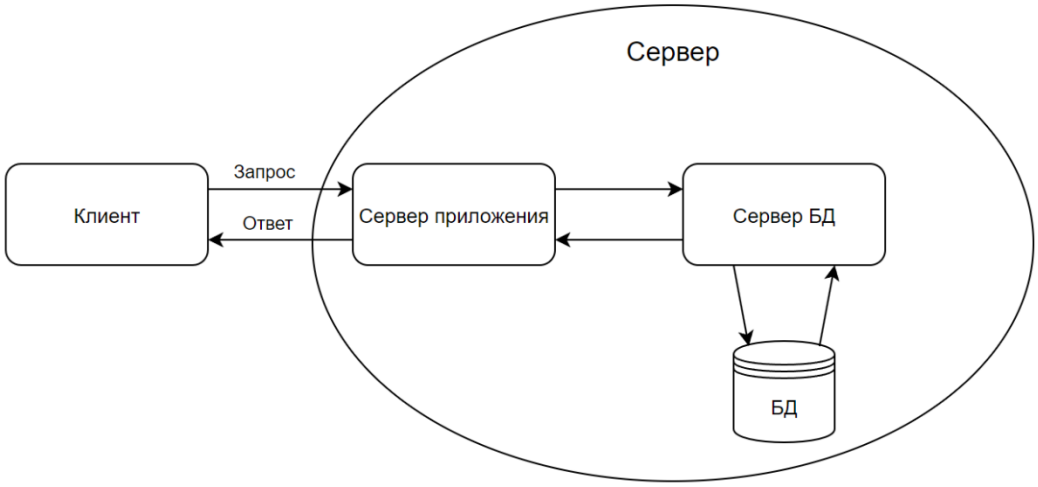

Рисунок 2.1 – Схема трёхзвенной клиент-серверной архитектуры

Клиентское звено, представленное веб-браузером пользователя, отправляет запросы на звено сервера приложений. Звено сервера приложений обрабатывает эти запросы, выполняет необходимую бизнес-логику и, при необходимости, взаимодействует со звеном сервера баз данных для получения или сохранения данных. После обработки запроса звено сервера приложений формирует ответ и отправляет его обратно клиентскому звену. Звено сервера баз данных отвечает исключительно за надёжное хранение и управление данными, предоставляя их по запросу звена сервера приложений.

Для организации кода на сервере приложений был выбран паттерн проектирования «Модель-Представление-Контроллер» (MVC) [6], схема которого показана на рисунке 2.2.

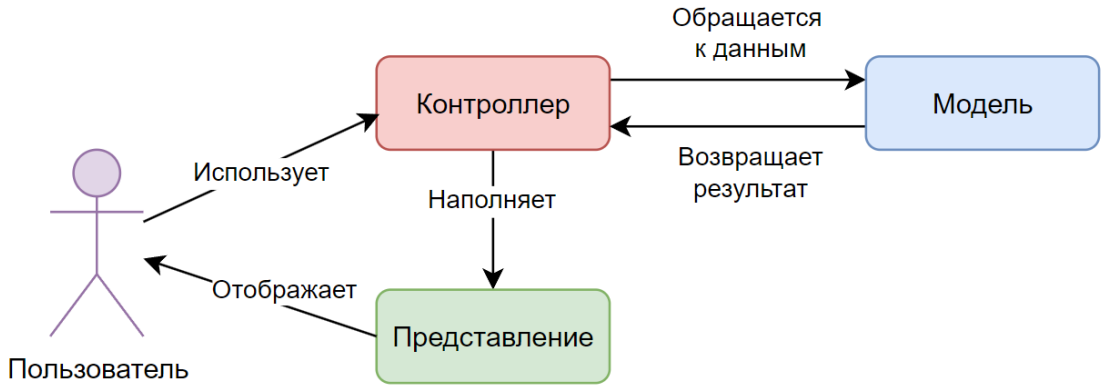

Рисунок 2.2 – Схема паттерна проектирования MVC

Выбор этого паттерна обусловлен его ключевым преимуществом — разделением ответственности. MVC позволяет чётко отделить логику работы с данными (Модель) от их визуального представления (Представление) и от логики обработки действий пользователя (Контроллер) [7].

Такое разделение делает код более структурированным, упрощает разработку, тестирование и поддержку приложения.

2.2 Выбор и обоснование используемых технологий

Структура и внешний вид страниц веб-приложения будут создаваться с использованием стандартов HTML5 и CSS3 [8]. HTML5 предоставляет семантическую разметку для контента [9], а CSS3 — широкие возможности для его стилизации [10]. Для ускорения разработки пользовательского интерфейса и обеспечения его адаптивности был использован CSS-фреймворк Bootstrap, который предоставляет готовую сетку и набор стилизованных компонентов, что позволило быстро создавать чистый и современный дизайн [10].

В рамках клиентской части интернет-ресурса также был использован JavaScript для реализации интерактивных элементов пользовательского интерфейса и обеспечения асинхронного взаимодействия с сервером, что улучшило пользовательский опыт [11].

Серверная часть веб-приложения реализована на языке PHP с использованием фреймворка Laravel. Выбор PHP обусловлен его широкой распространённостью в веб-разработке, а также наличием собственных знаний по работе с языком, полученных в рамках дисциплины «Проектирование и разработка серверных частей интернет-ресурсов» [12]. Выбор фреймворка Laravel обусловлен его строгим следованием паттерну MVC и богатой экосистемой [13].

В качестве СУБД была выбрана PostgreSQL [14]. Это мощная и надёжная СУБД, функциональность которой полностью покрывает требования проекта к хранению и обработке информации и с которой имеется личный опыт работы в рамках дисциплин «Проектирование баз данных» и «Разработка баз данных».

Для создания и управления средой разработки использовалась система контейнеризации Docker [15]. Она позволила создать отдельные контейнеры для сервера приложений и сервера базы данных, обеспечив изолированность и идентичность окружения на всех этапах разработки [16].

Для контроля версий исходного кода использовалась система Git. Она позволила отслеживать все изменения в проекте, откатываться к предыдущей версии при необходимости, а также вести историю разработки [17].

В качестве среды разработки был выбран Visual Studio Code, обусловленный его высокой производительностью и наличием богатой библиотеки расширений для работы со всеми выбранными инструментами [18].

2.3 Проектирование структуры базы данных

На этапе проектирования базы данных были выделены три ключевые сущности, необходимые для реализации функционала веб-приложения: «Пользователь», «Запись в расписании» и «Тег» [19].

Сущность «Пользователь» описывает пользователя приложения, который владеет своим расписанием. Она включает атрибуты: имя, адрес электронной почты и пароль.

Сущность «Запись в расписании» описывает отдельное событие в рамках недели. Она включает атрибуты: название, описание, день недели, время начала и время окончания.

Сущность «Тег» описывает метку для категоризации записей. Она включает атрибуты: название и цвет.

Между сущностями были определены следующие связи:

– каждый «Пользователь» может иметь множество «Записей в расписании»,

– каждый «Пользователь» может иметь множество «Тегов»,

– каждая «Запись в расписании» может быть помечена с помощью множества «Тегов»,

– каждым «Тегом» может быть помечено множество «Записей в расписании».

Для наглядного представления спроектированной структуры была построена ER-диаграмма в нотации Мартина [20] (рисунок 2.3).

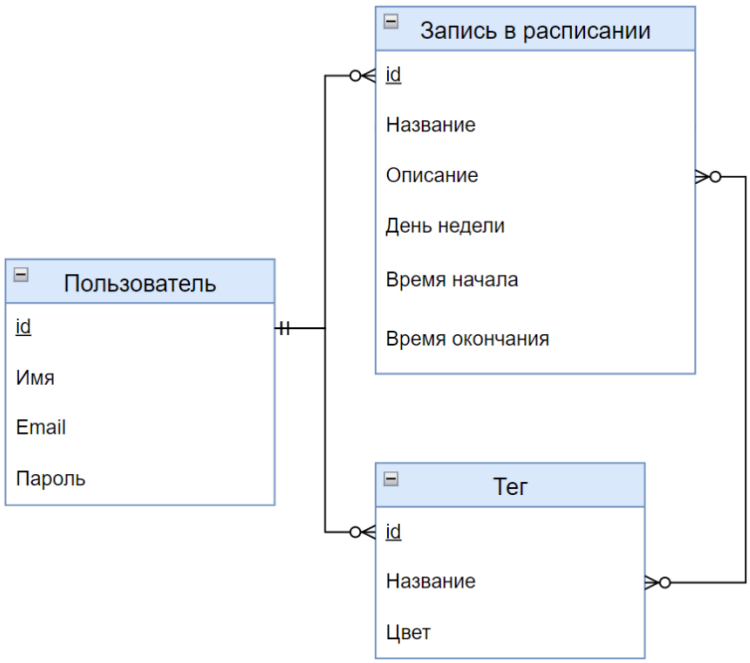

Рисунок 2.3 – ER-диаграмма базы данных

2.4 Проектирование пользовательского интерфейса и навигации

Путь пользователя начинается со страниц аутентификации (входа и регистрации), после прохождения которых он попадает на основную страницу для работы с расписанием. Навигация между страницей для работы с расписанием и страницей для работы с тегами, а также выход из системы, осуществляются через элементы меню, представленные в хедере приложения.

Создание, редактирование и удаление записей и тегов, реализованы через модальные окна. Это позволяет пользователю выполнять операции без перезагрузки страницы, что соответствует современным стандартам веб-разработки и улучшает общий пользовательский опыт [21].

Схема, иллюстрирующая основные страницы приложения и навигационные переходы между ними, представлена на рисунке 2.4.

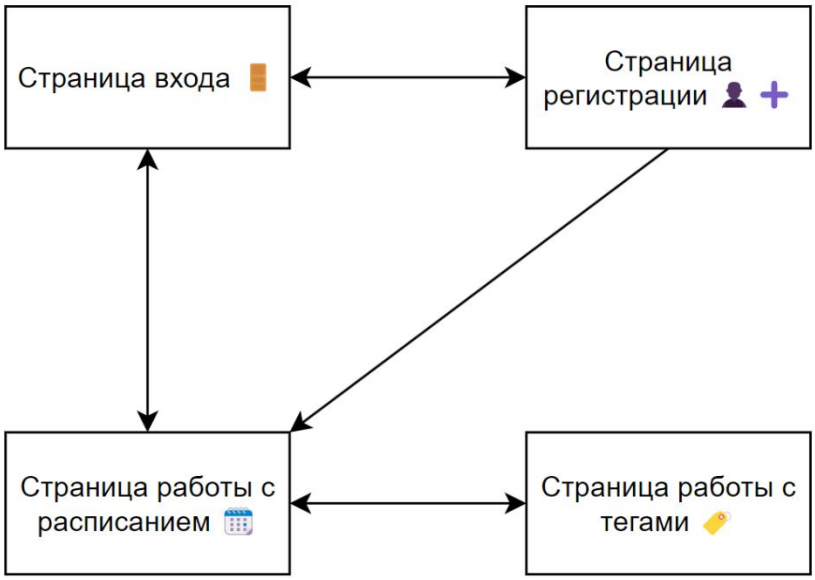

Рисунок 2.4 – Схема навигации между страницами веб-приложения

3 Разработка веб-приложения

3.1 Конфигурация среды выполнения

Реализация веб-приложения в соответствии с выбранной трёхзвенной архитектурой была осуществлена с использованием системы контейнеризации Docker. Это позволило изолировать сервер приложений и сервер базы данных PostgreSQL в отдельные контейнеры, обеспечив единообразие среды разработки.

Содержание файла compose.yaml с оркестрацией контейнеров для серверов представлено в приложении.

3.2 Реализация слоя базы данных

3.2.1 Миграции базы данных

Для управления структурой базы данных и обеспечения её версионного контроля в веб-приложении использовались миграции Laravel. Каждая миграция представляет собой файл, описывающий изменения в схеме БД, такие как создание или изменение таблиц. Это позволяет легко применять изменения к базе данных и, при необходимости, откатывать их до предыдущего состояния [22].

Фрагмент кода класса-миграции для создания таблицы записей в расписании представлен в листинге 3.1.

Листинг 3.1 — Фрагмента кода миграции по созданию таблицы записей в расписании

```
return new class extends Migration
{
    public function up(): void
    {
        Schema::create('schedule_entries', function (Blueprint $table) {
            $table->id();
            $table->foreignId('user_id')->constrained()->onDelete('cascade'); // Внешний ключ к таблице users
            $table->string('title');
            $table->text('description')->nullable();
```

Продолжение листинга 3.1

```
            $table->integer('day_of_week'); // 1 = Пн, 7 = Вс
            $table->time('start_time');
            $table->time('end_time');
            $table->timestamps();
        });

// Добавляем ограничения (check constraint) после создания таблицы
        DB::statement('ALTER TABLE schedule_entries ADD CONSTRAINT
check_end_time_gt_start_time CHECK (end_time > start_time)');
        DB::statement('ALTER TABLE schedule_entries ADD CONSTRAINT
check_day_of_week_range CHECK (day_of_week BETWEEN 1 AND 7)');
    }
    public function down(): void
    {
        Schema::dropIfExists('schedule_entries');
    }
};
```

Каждая миграция Laravel содержит два основных метода: up() и down(). Метод up() отвечает за применение изменений к базе данных, например, создание таблиц с помощью Schema::create(), где определяется их структура и поля. Метод down() предназначен для отката этих изменений. В некоторых случаях, для добавления специфичных ограничений базы данных, таких как CHECK ограничения, использовался DB::statement, позволяющий выполнять произвольные SQL-запросы.

3.2.2 Модели Eloquent

Для взаимодействия с данными на уровне приложения использовались модели Eloquent. Каждая модель представляет собой класс, реализующий паттерн Active Record и напрямую связанный с таблицей в базе данных. Это позволяет работать с записями как с объектами PHP, абстрагируясь от

написания прямых SQL-запросов и упрощая выполнение CRUD-операций [23]. В приложении были определены модели: User, ScheduleEntry, Tag.

Фрагмент кода класса-модели для записей в расписании представлен в листинге 3.2.

Листинг 3.2 — Фрагмент кода класса-модели “Schedule Entry”

```
class ScheduleEntry extends Model
{
    protected $fillable = [
        'user_id', 'title', 'description', 'day_of_week', 'start_time', 'end_time',
    ];

    // Получить пользователя, которому принадлежит запись расписания.
    public function user()
    {
        return $this->belongsTo(User::class);
    }

     // Получить все теги, связанные с этой записью.
    public function tags()
    {
        return $this->belongsToMany(Tag::class);
    }
}
```

Свойство $fillable определяет, какие атрибуты можно массово присваивать, обеспечивая защиту от уязвимостей. Для описания отношений между сущностями используются специальные методы, такие как belongsTo() для связи «многие-к-одному» и belongsToMany() для связи «многие-ко-многим», что позволяет легко извлекать связанные данные.

3.3 Реализация слоя серверной логики

3.3.1 Маршрутизация

Маршрутизация в Laravel является точкой входа для всех HTTP-запросов и отвечает за их направление на соответствующие методы контроллеров. Вся логика маршрутизации веб-приложения определена в файле routes/web.php. Этот файл действует как "карта" приложения, связывая URL-адреса и HTTP-методы с конкретными обработчиками, что позволяет централизованно управлять всеми доступными эндпоинтами.

Фрагмент кода файла маршрутизации представлен в листинге 3.3.

Листинг 3.3 — Фрагмент кода файла маршрутизации

```
// Маршруты аутентификации
Route::group([], function() {
    Route::get('login', [LoginController::class, 'showLoginForm'])->name('login');
    Route::post('login', [LoginController::class, 'login']);
    Route::post('logout', [LoginController::class, 'logout'])->name('logout');
    Route::get('register', [RegisterController::class, 'showRegistrationForm'])-
>name('register');
    Route::post('register', [RegisterController::class, 'register']);
});
// Группа маршрутов работы с бизнес-сущностями
// Требует аутентификации
Route::middleware(['auth'])->group(function () {
    // Маршруты для работы с ScheduleEntry
    Route::resource('schedule-entries', ScheduleEntryController::class)->only([
        'index', 'store', 'update', 'destroy'
    ]);
    // Маршруты для работы с Tag
    Route::resource('tags', TagController::class)->only([
        'index', 'store', 'update', 'destroy'
    ]);
});
```

Для определения маршрутов используется фасад Route. В приложении маршруты сгруппированы по их назначению. Маршруты, отвечающие за работу с записями расписания и тегами, объединены в группу с помощью метода group() и защищены посредником (middleware) auth, который обеспечивает доступ к ним только для аутентифицированных пользователей.

Для быстрой регистрации стандартных CRUD-маршрутов был использован метод resource(), который автоматически связывает стандартные URL с методами соответствующего ресурсного контроллера.

3.3.2 Валидация запросов

Для инкапсуляции логики валидации входящих данных и авторизации действий в приложении использовались классы запросов Laravel, наследуемые от FormRequest. Такой подход позволяет вынести сложную логику из контроллеров, делая их более лаконичными и читаемыми. В приложении были определены следующие классы валидации запросов: DestroyScheduleEntryRequest, DestroyTagRequest, StoreScheduleEntryRequest, StoreTagRequest, UpdateScheduleEntryRequest, UpdateTagRequest

Фрагмент кода класса для валидации запроса на обновление записи в расписании представлен в листинге 3.4.

Листинг 3.4 — Фрагмент кода класса валидации запроса на обновление записи в расписании

```
class UpdateScheduleEntryRequest extends FormRequest
{
    public function authorize(): bool
    {
        // Получаем запись расписания из параметров маршрута
        $schedule_entry = $this->route('schedule_entry');

        // Проверяем, что запись существует и принадлежит текущему пользователю
        return $schedule_entry && $schedule_entry->user_id === auth()->id();
    }
```

Продолжение листинга 3.4

```
    public function rules(): array
    {
        return [
            'title' => ['required', 'string', 'max:255'],
            'description' => ['nullable', 'string'],
            'day_of_week' => ['required', 'integer', 'between:1,7'],
            'start_time' => ['required', 'date_format:H:i'],
            'end_time' => ['required', 'date_format:H:i', 'after:start_time'],
            'tags' => ['nullable', 'array'],
            'tags.*' => ['integer', Rule::exists('tags', 'id')->where('user_id', auth()->id())],
        ];
    }
}
```

Каждый класс запроса содержит два основных метода: authorize() и rules().

Метод authorize() определяет, имеет ли пользователь право выполнять данный запрос. В примере он получает редактируемую запись расписания из параметров маршрута и проверяет, что её user_id соответствует id текущего пользователя, полученному с помощью фасада Auth, созданного для простого и лаконичного доступа к функционалу аутентификации.

Метод rules() возвращает массив правил валидации для полей формы, обеспечивая корректность и целостность данных перед их обработкой в контроллере. Например, для запроса на обновление записи в расписании, здесь определено ограничение: день недели должен быть числом из диапазона [1, 7].

3.3.3 Контроллеры

Контроллеры в архитектуре MVC являются центральным звеном, обрабатывающим HTTP-запросы. Они отвечают за координацию действий, обращаясь к моделям для работы с данными и выбирая представления для формирования ответа, который может быть как HTML-страницей, так и JSON-

объектом. В приложении были определены следующие контроллеры: LoginController, RegisterController, ScheduleEntryController, TagController.

В качестве первого примера рассмотрим метод index() контроллера ScheduleEntryController, который отвечает за отображение основной страницы с расписанием. Его код представлен в листинге 3.5.

Листинг 3.5 — Код метода index() контроллера ScheduleEntryController

```
public function index(Request $request)
{
    $query = auth()->user()->scheduleEntries()->with('tags');

    // Фильтрация по дню недели
    if ($request->has('day') && $request->query('day') !== 'all') {
        $query->where('day_of_week', $request->query('day'));
    }
    // Поиск по названию и описанию
    if ($request->filled('search')) {
        $searchTerm = $request->query('search');
        $query->where(function ($subQuery) use ($searchTerm) {
            $subQuery->where('title', 'ILIKE', "%{$searchTerm}%")
                    ->orWhere('description', 'ILIKE', "%{$searchTerm}%");
        });
    }
    // Фильтрация по тегам
    if ($request->filled('tags')) {
        $tagIds = $request->query('tags');
        if (is_array($tagIds) && !empty($tagIds)) {
            $validatedTagIds = array_filter($tagIds, 'is_numeric');
            if (!empty($validatedTagIds)) {
                $query->whereHas('tags', function ($q) use ($validatedTagIds) {
                    $q->whereIn('tags.id', $validatedTagIds);
                });
            }
        }
    }
```

Продолжение листинга 3.5

```
        $entries = $query->orderBy('day_of_week')->orderBy('start_time')->get();

        $groupedEntries = $entries->groupBy('day_of_week');
        $tags = auth()->user()->tags()->get();
        //
        return view('schedule-entries.index', [
            'groupedEntries' => $groupedEntries,
            'tags' => $tags
        ]);
    }
```

Метод index() динамически формирует запрос к базе данных с помощью конструктора запросов Eloquent, учитывая параметры фильтрации по дню недели, поисковые запросы и выбранные теги. После получения и группировки данных, он передает их в Blade-шаблон schedule-entries.index, который генерирует и возвращает HTML-страницу пользователю.

Второй пример демонстрирует обработку AJAX-запроса на обновление существующей записи в методе update() того же контроллера. Его код представлен в листинге 3.6.

Листинг 3.6 — Код метода update() контроллера ScheduleEntryController

```
    public function update(UpdateScheduleEntryRequest $request, ScheduleEntry
$schedule_entry)
    {
        $validated = $request->validated();
        $schedule_entry->update($validated);

        // Прикрепляем теги, если они были переданы
        if (isset($validated['tags'])) {
            $schedule_entry->tags()->sync($validated['tags']);
        } else {
```

Продолжение листинга 3.6

```
        // Если теги не переданы (например, все были сняты), отсоединяем все теги
        $schedule_entry->tags()->detach();
    }

    // Формируем успешный ответ с URL для редиректа.
    $redirectUrl = $this->getBackUrl($this->fallbackRoute);
    return JsonResponseBuilder::success('Запись успешно обновлена.', ['redirectUrl' =>
$redirectUrl]);
  }
```

Метод update() использует механизм внедрения зависимостей Laravel для автоматической валидации входящих данных через класс запроса UpdateScheduleEntryRequest и получения экземпляра модели ScheduleEntry. После успешной валидации и обновления данных, контроллер возвращает JSON-ответ с подтверждением операции и URL для клиентского редиректа, что обеспечивает плавное взаимодействие без перезагрузки страницы.

3.4 Реализация слоя клиентского представления

3.4.1 Шаблоны Blade

HTML-структура слоя представления веб-приложения реализована с помощью шаблонизатора Blade. Этот инструмент Laravel используется для генерации динамических HTML-страниц и позволяет отделить логику отображения от бизнес-логики, используя простой синтаксис для работы с данными, переданными из контроллера.

Фрагмент кода, отвечающий за отображение записей расписания в виде таблицы, представлен в листинге 3.4.1.1.

Листинг 3.7 — Фрагмент кода шаблона Blade, отвечающий за отображение записей расписания в виде таблицы

```
<tbody>
    @php $lastDay = null; @endphp
    @forelse ($groupedEntries as $day => $entries)
        @foreach ($entries as $entry)
            <tr class="schedule-row" id="entry-{{ $entry->id }}" data-entry="{{
json_encode($entry->load('tags')) }}" data-bs-toggle="modal" data-bs-
target="#editScheduleEntryModal" style="cursor: pointer;">
                <td>
                    @if ($day !== $lastDay)
                        <strong>{{ $days[$day] }}</strong>
                    @endif
                </td>
                <td>{{ date('H:i', strtotime($entry->start_time)) }} - {{ date('H:i', strtotime($entry-
>end_time)) }}</td>
                <td>{{ $entry->title }}</td>
                <td>{{ $entry->description }}</td>
                <td>
                    @foreach ($entry->tags as $tag)
                        <span class="badge text-bg-{{ ColorMapper::colorToBsClass($tag->color)
}}">{{ $tag->name }}</span>
                    @endforeach
                </td>
            </tr>
            @php $lastDay = $day; @endphp
        @endforeach
    @empty
        <tr>
            <td colspan="5" class="text-center">
                @if (request()->filled('search') || request()->filled('tags'))
                    Записи, удовлетворяющие условиям фильтра, не найдены.
                @else
                    @if (request('day') && request('day') !== 'all')
                        На выбранный день записей нет.
```

Продолжение листинга 3.7

```
                @else
                    В вашем расписании пока нет записей.
                @endif
            @endif
        </td>
    </tr>
  @endforelse
</tbody>
```

В приведенном фрагменте кода директива @forelse используется для итерации по массиву $groupedEntries, который передается в представление из контроллера. Эта директива обрабатывает как случай наличия записей, так и их отсутствия, выводя сообщение внутри блока @empty. Для улучшения читаемости таблицы условная директива @if применяется для отображения названия дня недели только для первой записи в группе. Безопасный вывод данных обеспечивается синтаксисом {{ }}, а полные данные о записи кодируются в JSON-формат и сохраняются в data-атрибуте для последующего использования клиентским кодом.

3.4.2 Клиентская логика (JavaScript)

Клиентская логика, реализованная на языке JavaScript, отвечает за интерактивность пользовательского интерфейса и асинхронное взаимодействие с сервером. Такой подход, известный как AJAX, позволяет выполнять операции с данными без полной перезагрузки страницы, что значительно улучшает пользовательский опыт.

Код, демонстрирующий отправку формы для создания новой записи в расписании, представлен в листинге 3.8.

Продолжение листинга 3.8

```
document.addEventListener('DOMContentLoaded', () => {
    const createForm = document.getElementById('createScheduleEntryForm');
    const errorContainer = document.getElementById('create-error-container');

    if (createForm) {
        createForm.addEventListener('submit', function (event) {
            event.preventDefault();
            const formData = new FormData(this);
            // Собираем выбранные теги и добавляем их в FormData
            createForm.querySelectorAll('input[name="tags[]"]:checked').forEach(checkbox =>
{
                formData.append('tags[]', checkbox.value);
            });
            axios.post(this.action, formData)
                .then(response => {
                    // В случае успеха, используем URL для редиректа из ответа сервера
                    if (response.data?.payload?.redirectUrl) {
                        window.location.href = response.data.payload.redirectUrl;
                    } else {
                        // Если URL не пришел, просто перезагружаем страницу
                        window.location.reload();
                    }
                })
                .catch(error => handleResponseError(error, errorContainer));
        });
    }
...
});
```

В приведенном фрагменте кода показан обработчик события отправки формы. Он перехватывает стандартное действие браузера с помощью event.preventDefault() и собирает все данные формы в объект FormData. Затем,

с помощью библиотеки axios, выполняется асинхронный POST-запрос на сервер [24].

Ключевой особенностью является обработка успешного ответа: клиентский код не принимает решений о дальнейших действиях, а ожидает инструкций от контроллера. В случае успеха сервер возвращает JSON-ответ, содержащий URL для перенаправления, и клиентский скрипт лишь исполняет это указание. Таким образом, ответственность за навигацию остается на серверной стороне, а слой представления отвечает только за исполнение команд.

4 Функциональное тестирование веб-приложения

4.1 Ход тестирования

Для проверки корректности работы веб-приложения было проведено ручное функциональное тестирование методом «черного ящика» [25]. Этот подход позволяет оценить функциональность системы с точки зрения конечного пользователя, не углубляясь в детали внутренней реализации. Основной целью тестирования являлась демонстрация работоспособности ключевых функций, в частности, CRUD-операций над записями расписания.

Тестирование началось с обращения к веб-приложению через браузер, после чего отобразилась страница входа (рисунок 4.1).

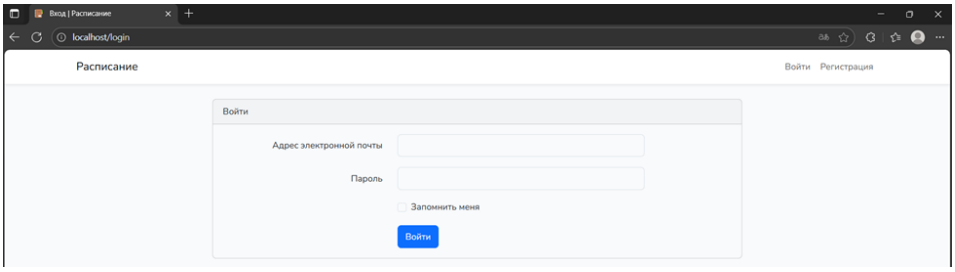

Рисунок 4.1 – Отображение страницы входа

С помощью элемента навигации в хедере совершён переход на страницу и регистрации с вводом данных для создания нового пользователя (рисунок 4.2).

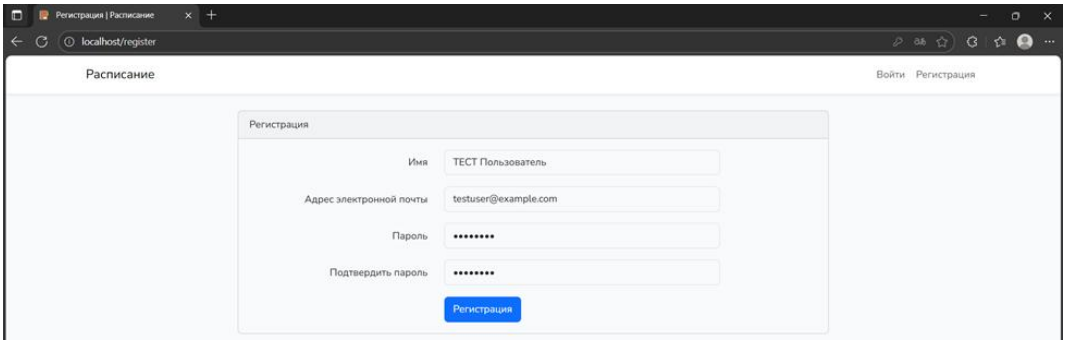

Рисунок 4.2 – Отображение страницы регистрации

После нажатия кнопки «Регистрация» произошло успешное создание нового пользователя и веб-приложение перенаправило на главную страницу,

где отобразился интерфейс для просмотра и управления расписанием (рисунок 4.3).

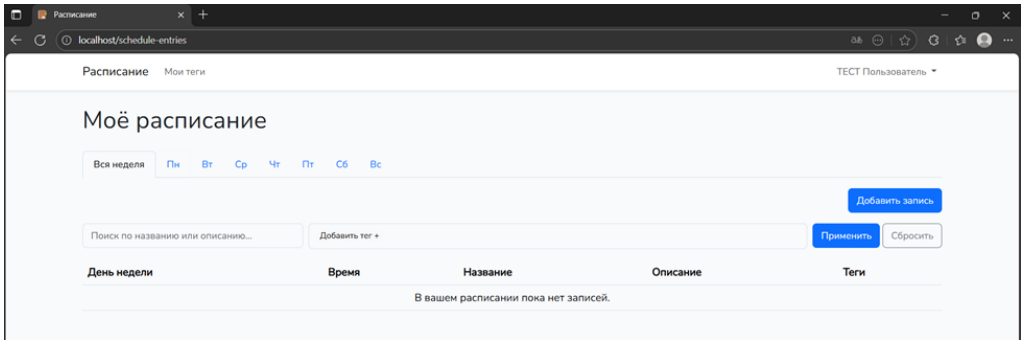

Рисунок 4.3 – Отображение страницы управления расписанием

С помощью элемента навигационного меню в хедере веб-приложения был осуществлен переход на страницу управления тегами (рисунок 4.4).

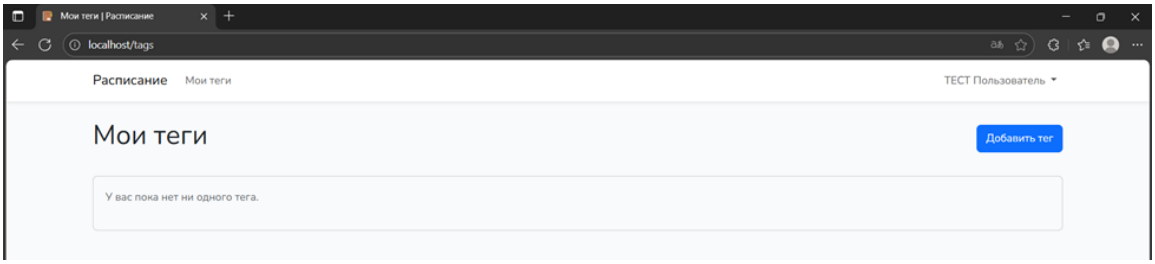

Рисунок 4.4 — Отображение страницы управления тегами

Далее, нажав на кнопку «Добавить тег», было вызвано модальное окно с формой. После заполнения полей формы данными для нового тестового тега и нажатия кнопки «Сохранить», страница обновилась, и созданный тег был корректно отображен в общем списке (рисунки 4.4–4.5).

Рисунок 4.4 — Тестирование добавления тега

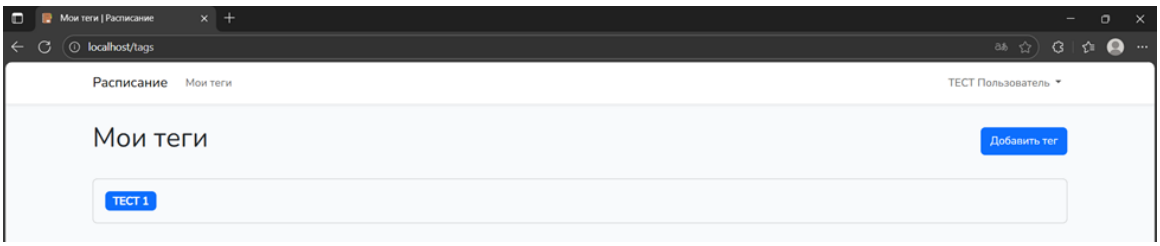

Рисунок 4.5 — Успешное добавление тега

Затем был осуществлен возврат на страницу работы с расписанием. Нажав на кнопку «Добавить запись», было открыто модальное окно с формой. При заполнении формы данные были намеренно введены некорректно: не был выбран ни один день недели, а время начала было указано позднее времени окончания. После нажатия кнопки «Сохранить» в модальном окне появились сообщения об ошибках валидации, что подтвердило корректную работу серверной валидации данных и правильную обработку ответов с ошибками на клиентской стороне (рисунок 4.6).

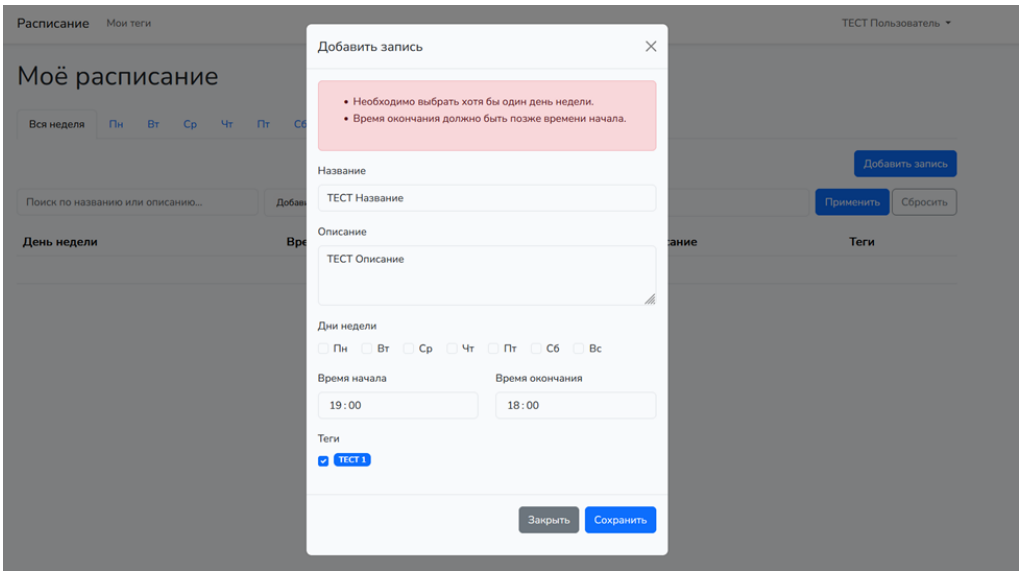

Рисунок 4.6 — Демонстрация валидации данных

После ввода корректных данных и указания тестового тега для записей, форма была успешно отправлена, страница обновилась и все созданные записи корректно отобразились в расписании (рисунки 4.7–4.8).

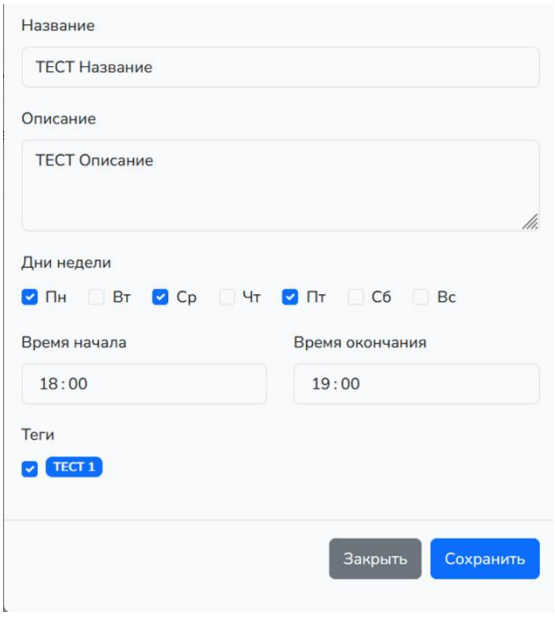

Рисунок 4.7 – Тестирование добавления новых записей

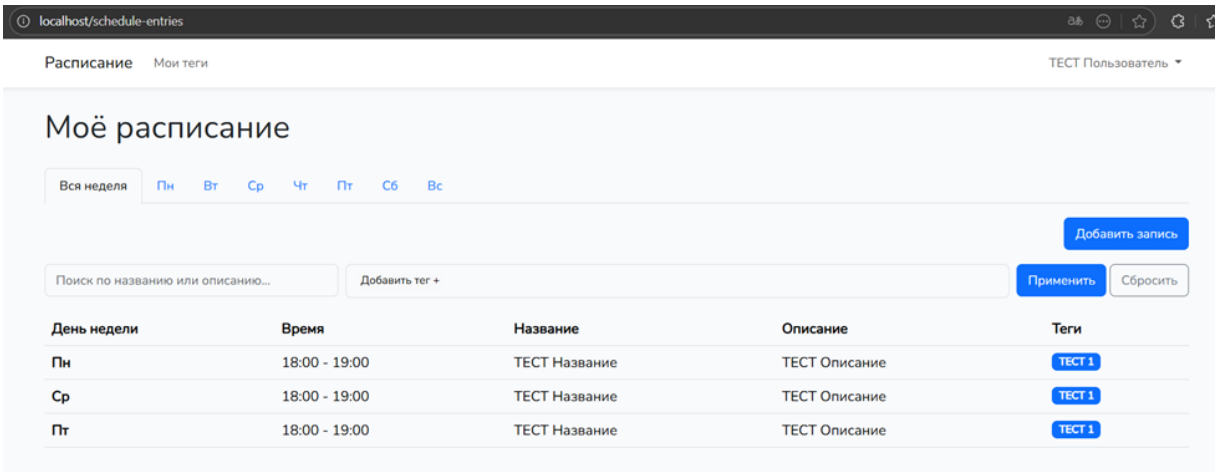

Рисунок 4.8 – Корректное отображение добавленных записей

Далее было проведено тестирование изменения записи в расписании, включая открепление тега. Изменённая запись корректно отобразилась в расписании (рисунки 4.9–4.10).

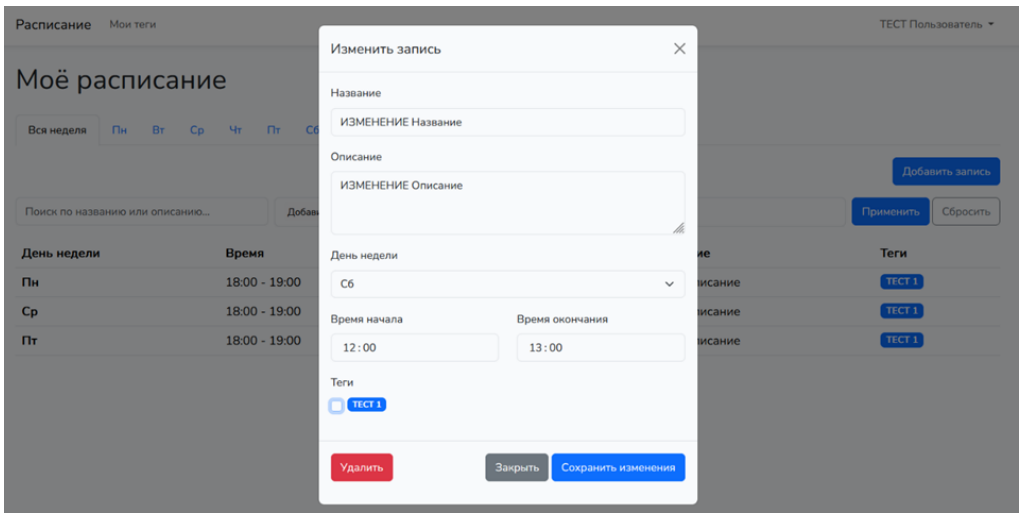

Рисунок 4.9 – Тестирование изменения записи

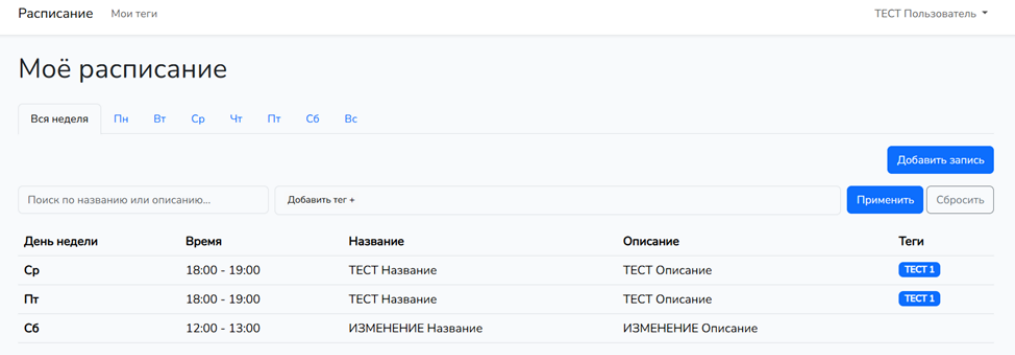

Рисунок 4.10 – Корректное отображение изменённой записи

Затем было проведено тестирование функционала удаление записей. При попытке удаления одной из созданных ранее записей появилось модальное окно подтверждения удаления. После подтверждения запись была корректно удалена и перестала отображаться в интерфейсе (рисунки 4.11–4.12).

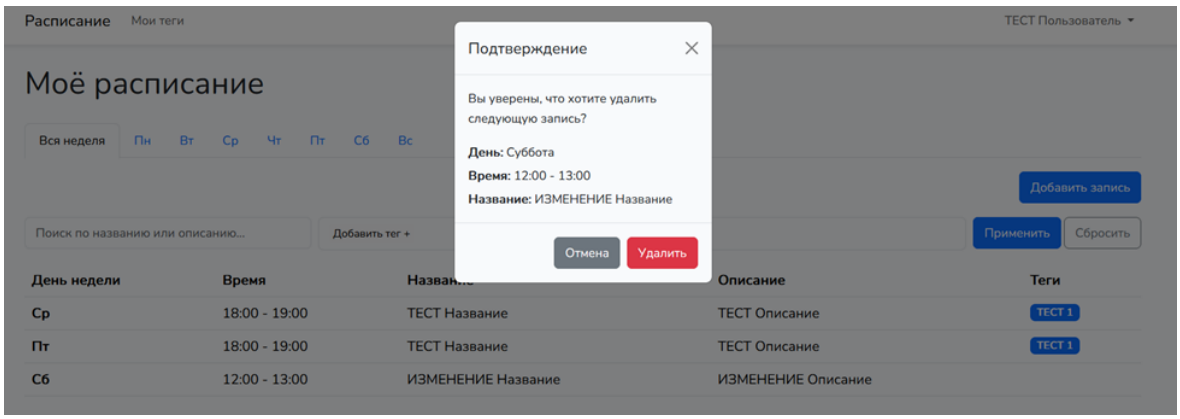

Рисунок 4.11 — Тестирование подтверждения удаления

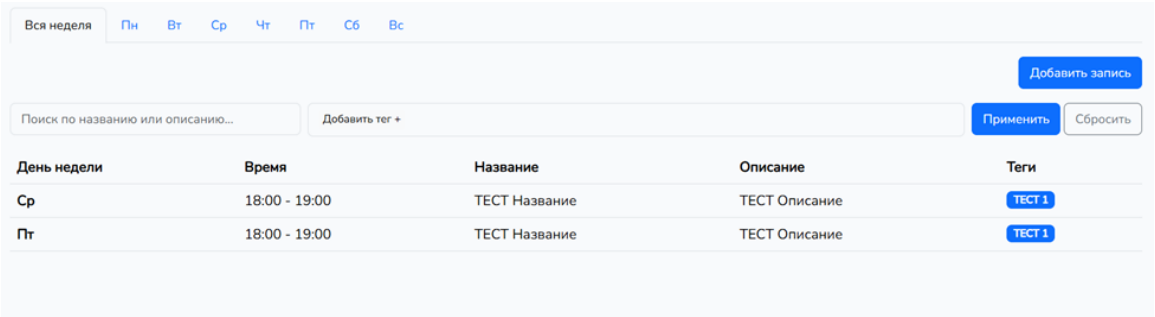

Рисунок 4.12 — Корректное отображение расписания после удаления записи

Далее было проведено тестирование редактирования тега, которое включало изменение названия и цвета (рисунок 4.13).

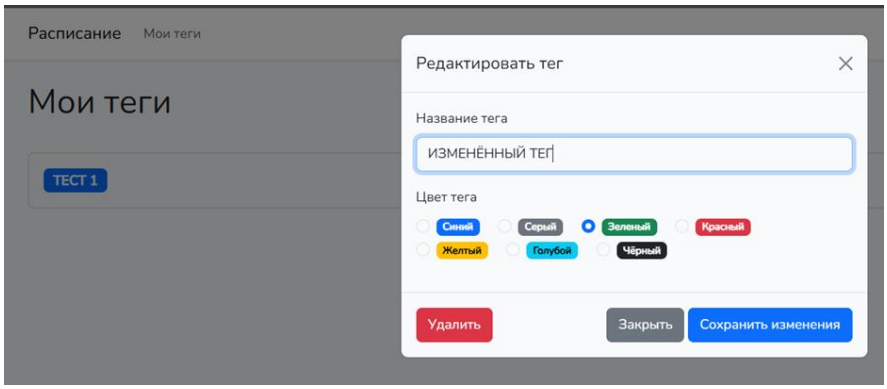

Рисунок 4.13 — Тестирование изменения тега

Интерфейс корректно отобразил изменение тега на странице тегов (рисунок 4.14).

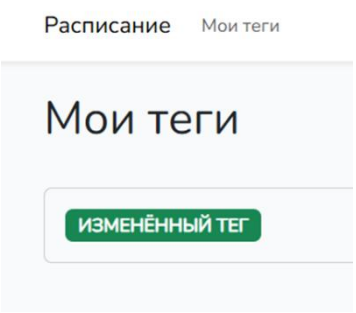

Рисунок 4.14 — Корректное отображение изменённого тега на странице тегов

Далее на странице управления расписанием было проверено, что тег был корректно обновлён у записей, к котором он прикреплён, а также был протестирован функционал отображения записей только на конкретный день недели (рисунок 4.15)

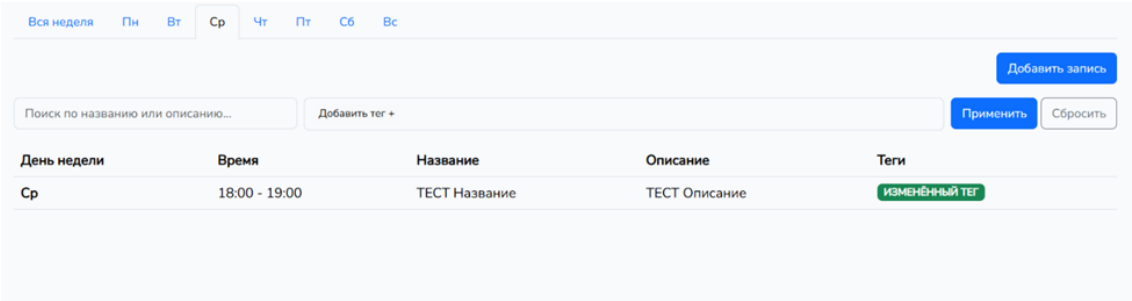

Рисунок 4.15 — Корректное отображение изменённого тега на странице расписание и корректная фильтрация по дню недели

Затем было протестировано удаление тега. Как и ожидалось, после подтверждения удаления тег пропал со страницы управления тегами, а также был открепён от записей, к которым был прикреплён (рисунки 4.16–4.18).

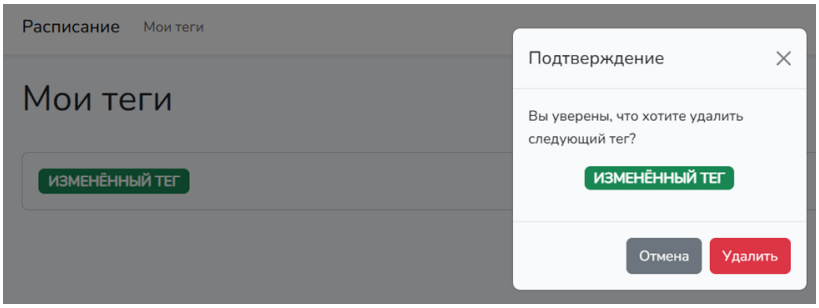

Рисунок 4.16 — Тестирование удаления тега

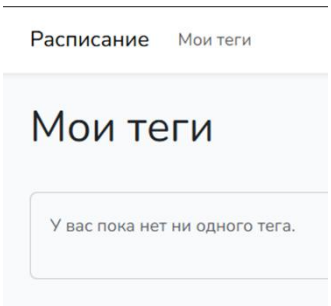

Рисунок 4.17 — Корректное отображение страницы тегов после удаления тега

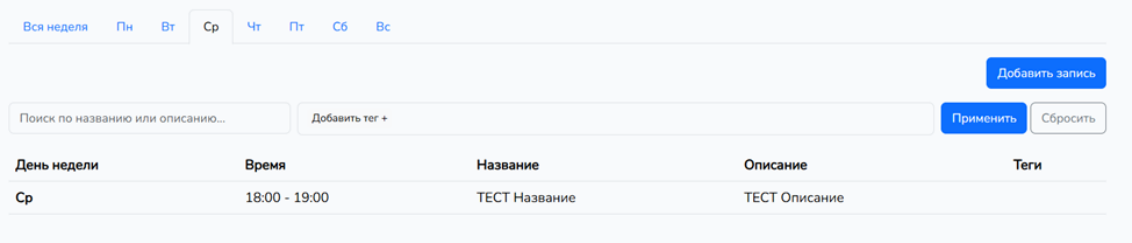

Рисунок 4.18 — Корректное отображение страницы расписания после удаления тега

4.2 Итоги тестирования

По результатам выполнения тестирования подтверждена корректная работа основного функционала веб-приложения, связанного с управлением записями в расписании и тегами, аутентификацией и межстраничной навигацией. Все продемонстрированные CRUD-операции выполняются без ошибок, что свидетельствует о базовой работоспособности ключевых возможностей системы.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была спроектирована и разработана серверная часть веб-приложения «Расписание». На основе проведенного анализа предметной области была обоснована актуальность создания простого и удобного инструмента для управления личным недельным расписанием, что выгодно отличает его от рассмотренных аналогов.

Результатом работы является веб-приложение с межстраничной навигацией и внешним видом страниц, соответствующим современным стандартам веб-разработки, позволяющее пользователям проходить аутентификацию, управлять записями своего расписания и гибко настраивать их с помощью персонализированных тегов. Реализованный функционал включает создание, редактирование и удаление записей, а также их фильтрацию по различным критериям, что было подтверждено в ходе функционального тестирования.

Данный проект позволил получить практический опыт в разработке полноценного веб-приложения, начиная от проектирования базы данных на основе ER-диаграммы и реализации серверной бизнес-логики с помощью фреймворка Laravel, до создания интерактивного пользовательского интерфейса.

Отчет по курсовой работе выполнен в соответствии с ГОСТ 7.32-2017 [26]. Для демонстрации проделанной работы подготовлена презентация. Все поставленные задачи выполнены. Цель работы достигнута.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Бесплатная программа для составления расписания | Visme [Электронный ресурс] – URL: https://www.visme.co/ru/sozdat-raspisanie-online/ (дата обращения 11.11.2025).

2. Planzeo – Бесплатный онлайн-генератор расписания занятий [Электронный ресурс] – URL: https://planzeo.com/ru/ (дата обращения 11.11.2025).

3. Генератор расписаний - создание оптимальных расписаний для учебных заведений | Gensio [Электронный ресурс] – URL: https://gensio.ru/tools/time/schedule-generator (дата обращения 11.11.2025).

4. Использование диаграммы вариантов использования UML при проектировании программного обеспечения / Хабр [Электронный ресурс] – URL: https://habr.com/ru/articles/566218/ (дата обращения 11.11.2025).

5. Волков М. Ю. Разработка серверных частей интернет-ресурсов : учебное пособие / М. Ю. Волков, В. В. Литвинов, А. А. Лобанов. — Москва : РТУ МИРЭА, 2021. — 188 с.

6. Что такое концепция Model, View, Controller и как она работает [Электронный ресурс] – URL: https://education.yandex.ru/journal/model-view-controller (дата обращения 11.11.2025).

7. Что такое MVC и как его использовать / Skillbox Media [Электронный ресурс] – URL: https://skillbox.ru/media/code/chto_takoe_mvc_bazovye_kontseptsii_i_primer_prilozheniya/ (дата обращения 11.11.2025).

8. Robbins J. Learning Web Design: A Beginner's Guide to HTML, CSS, JavaScript, and Web Graphics – O'Reilly Media – 2012. - 624 p.

9. HTML: HyperText Markup Language | MDN [Электронный ресурс] – URL: https://developer.mozilla.org/en-US/docs/Web/HTML (дата обращения 11.11.2025).

10. CSS: Cascading Style Sheets | MDN [Электронный ресурс] – URL: https://developer.mozilla.org/en-US/docs/Web/CSS (дата обращения 11.11.2025).

11. JavaScript | MDN [Электронный ресурс] – URL: https://developer.mozilla.org/ru/docs/Web/JavaScript (дата обращения 11.11.2025).

12. Основной учебник PHP для новичков [Электронный ресурс] – URL: https://code.mu/ru/php/book/prime/ (дата обращения 11.11.2025).

13. Laravel — экосистема, а не просто PHP-фреймворк / Хабр [Электронный ресурс] – URL: https://habr.com/ru/articles/334776/ (дата обращения 11.11.2025).

14. PostgreSQL: Documentation: 18: 1. What Is PostgreSQL? [Электронный ресурс] – URL: https://www.postgresql.org/docs/current/intro-whatis.html (дата обращения 11.11.2025).

15. Разработка Laravel в Docker [Электронный ресурс] – URL: https://purpleschool.ru/knowledge-base/article/laravel (дата обращения 11.11.2025).

16. Dockerize a Laravel 11 app - DEV Community [Электронный ресурс] – URL: https://dev.to/frankalvarez/dockerize-a-laravel-11-app-4g4a (дата обращения 11.11.2025).

17. Git: что это такое и как пользоваться им в 2025 году [Электронный ресурс] – URL: https://skyeng.ru/it-industry/it/git-chto-eto-i-kak-im-polzovatsya-v-2025-godu/ (дата обращения 11.11.2025).

18. Редактор кода Visual Studio Code. Самый подробный гайд по настройке и установке плагинов для начинающих / Хабр [Электронный ресурс] – URL: https://habr.com/ru/articles/490754/ (дата обращения 11.11.2025).

19. Бизнес-сущности и информационная модель [Электронный ресурс] – URL: https://edu.tsu.ru/eor/resourse/865/html/25.html (дата обращения 11.11.2025).

20. ER-диаграммы: что это, применение, нотации - как создать ER-диаграмму сущность-связь, примеры [Электронный ресурс] – URL: https://practicum.yandex.ru/blog/chto-takoe-er-diagramma/ (дата обращения 11.11.2025).

21. Improving User Experience with AJAX [Электронный ресурс] – URL: Improving User Experience with AJAX - Toxigon (дата обращения 11.11.2025).

22. Database: Migrations - Laravel 12.x - The PHP Framework For Web Artisans [Электронный ресурс] – URL: https://laravel.com/docs/12.x/migrations (дата обращения 11.11.2025).

23. Eloquent: Getting Started - Laravel 12.x - The PHP Framework For Web Artisans [Электронный ресурс] – URL: https://laravel.com/docs/12.x/eloquent (дата обращения 11.11.2025).

24. Eloquent: Getting Started - Laravel 12.x - The PHP Framework For Web Artisans [Электронный ресурс] – URL: https://laravel.com/docs/12.x/eloquent (дата обращения 11.11.2025).

25. What Is Axios? - GeeksforGeeks [Электронный ресурс] – URL: https://www.geeksforgeeks.org/html/what-is-axios/ (дата обращения 11.11.2025).

26. ГОСТ 7.32-2017 | НАЦИОНАЛЬНЫЕ СТАНДАРТЫ [Электронный ресурс] – URL: https://protect.gost.ru/document.aspx?control=7&id=218998 (дата обращения 11.11.2025).

ПРИЛОЖЕНИЕ

Содержание файла compose.yaml с оркестрацией контейнеров

```
services:
  laravel.test:
    build:
      context: './vendor/laravel/sail/runtimes/8.4'
      dockerfile: Dockerfile
      args:
        WWWGROUP: '${WWWGROUP}'
    image: 'sail-8.4/app'
    extra_hosts:
      - 'host.docker.internal:host-gateway'
    ports:
      - '${APP_PORT:-80}:80'
      - '${VITE_PORT:-5173}:${VITE_PORT:-5173}'
    environment:
      WWWUSER: '${WWWUSER}'
      LARAVEL_SAIL: 1
      XDEBUG_MODE: '${SAIL_XDEBUG_MODE:-off}'
      XDEBUG_CONFIG: '${SAIL_XDEBUG_CONFIG:-client_host=host.docker.internal}'
      IGNITION_LOCAL_SITES_PATH: '${PWD}'
    volumes:
      - '.:/var/www/html'
    networks:
      - sail
    depends_on:
      - pgsql
```

```
    pgsql:
        image: 'postgres:18-alpine'
        ports:
            - '${FORWARD_DB_PORT:-5432}:5432'
        environment:
            PGPASSWORD: '${DB_PASSWORD:-secret}'
            POSTGRES_DB: '${DB_DATABASE}'
            POSTGRES_USER: '${DB_USERNAME}'
            POSTGRES_PASSWORD: '${DB_PASSWORD:-secret}'
        volumes:
            - 'sail-pgsql:/var/lib/postgresql/data'
            - './vendor/laravel/sail/database/pgsql/create-testing-database.sql:/docker-entrypoint-initdb.d/10-create-testing-database.sql'
        networks:
            - sail
        healthcheck:
            test:
                - CMD
                - pg_isready
                - '-q'
                - '-d'
                - '${DB_DATABASE}'
                - '-U'
                - '${DB_USERNAME}'
            retries: 3
            timeout: 5s
networks:
    sail:
        driver: bridge
volumes:
    sail-pgsql:
        driver: local
```